

HW6 Key Points

Priority queue

Each time pop the object which has the highest priority.

Implementation

- Multiple queues
 - AVL Trees
 - Heaps
-

Binary Heap

Definition(Min-Heap): 1.The key associated with the root is less than or equal to the keys associated with the sub-trees (if any) 2.The sub-trees (if any) are also min-heaps

Operations: Top, Pop, Push

Percolation: the lighter (smaller) objects move up from the bottom of the min-heap

Q:How to keep balance?

A: AVL Trees, B-Trees, Red-black Trees, and **Complete Binary Trees**

In **Complete Binary Trees**: **Push:** place the new node as a leaf node in the appropriate location and percolate up. **Pop:** replace the root with the last entry in the heap, then percolate down.

Array Implementation

Start at index 1. For a node k ,

```
parent = k >> 1;
left_child = k << 1;
right_child = left_child | 1;
```

Runtime Analysis

	front	arbitrary	back
insert	$O(\ln(n))^*$	$O(1)$	$O(1)$
access	$O(1)$	$O(n)$	$O(n)$
delete	$O(\ln(n))$	$O(n)$	$O(n)$

Floyd's Method

Build a heap in $O(n)$ time.

Heap Sort

Use a Max-Heap, array implementation, inplace.

Method: 1. Use Floyd's Method to build a Max-Heap. 2. Repeatedly Pop the maximum and put it at the end of the array.

$O(n \log n)$ of time, $O(1)$ of additional space.

Q: How to keep the original order in a priority queue?

A: Use a pair (n, k) to represent each element, where n is the value, k is its rank to be inserted.

Huffman Coding

Create binary tree nodes with character and frequency of each character

Place nodes in a priority queue. –The lower the occurrence, the higher the priority in the queue

Method: Each time select the least two frequent trees and merge them. Till there is only one left.

Binary Search Tree

Properties

- all objects in the left sub-tree to be less than the object stored in the root node

- all objects in the right sub-tree to be greater than the object in the root object
- the two sub-trees are themselves binary search trees

Operations

- member variables: `element, left_tree, right_tree`
- **Find the Max/Min:** $O(h)$
- **Find:** To determine membership, traverse the tree based on the linear relationship. $O(h)$
- **Insert:** Step through down the tree. If we find the object already in the tree, we will return. Otherwise, we will arrive at an empty node. Insert there. $O(h)$
- **Erase:** i) no child. ii) only one child, promote the sub-tree. iii) two children, Replace it with the minimum object in the right sub-tree; Erase that object from the right sub-tree. $O(h)$
- **Next:** If the node has a right sub-tree, the minimum object in that sub-tree is the next object. Otherwise, it is the first larger object (if any) that exists in the path from the node to the root. $O(h)$
- **Finding the k -th:** If the left-sub-tree has $l = k$ entries, return the current node. If the left sub-tree has $l > k$ entries, return the k -th entry of the left sub-tree. Otherwise, the left sub-tree has $l < k$ entries, so return the $(k - l - 1)$ -th entry of the right sub-tree

Runtime

Almost all of the relevant operations on a binary search tree are $O(h)$. Our goal will be to find tree structures where we can maintain a height of $O(\ln(n))$.

Solution: AVL trees, Red-black trees, B trees, B+ trees, Splay trees.....
